

BexFLOW — *technische Übergabe*

Eine Schweizer Pipeline von der Bankauszugs-Datei bis zur gebuchten Sammelbuchung in be-xio. Dieses Dokument richtet sich an die Entwicklerin oder den Entwickler, die den Code übernimmt: vom Produktverständnis bis zum Live-Betrieb.

Codebasis /opt/bp-pilot/app

Interner Codename BP Pilot

Sprache Python 3.11 · TypeScript

Runtime Docker Compose · Ubuntu

Live unter app.bexflow.ch · app.bp-pilot.ch

BACKEND

FastAPI · ~8 kLoC
· 65 Routen

FRONTEND

Next.js 14 ·
React 18

DATENHALTUNG

PostgreSQL 16
+ MinIO

KI-ANBIETER

Infomaniak
(OpenAI-kompatibel)

§01 ÜBERSICHT Was BexFLOW leistet

BexFLOW (intern *BP Pilot*) ist eine Schweizer B2B-Webanwendung, die sich zwischen die Bankauszüge eines Unternehmens und dessen Buchhaltungssystem *bexio* stellt. Sie strukturiert und beschleunigt die Erfassung am Monatsende und verhindert Doppelbuchungen — die buchhalterische Entscheidung pro Zeile trifft weiterhin die Nutzerin.

Zielnutzer sind kleine und mittlere Schweizer Unternehmen, Treuhänderinnen und interne Buchhalter. Sie melden sich an, verbinden *bexio* einmalig per OAuth und laden anschliessend das CSV oder XLSX hoch, das sie bei ihrer Bank exportiert haben (UBS, PostFinance, Raiffeisen, Revolut usw.).

Was die Nutzerin sieht

1. **Upload.** Bankauszug per Drag-and-Drop in den Browser ziehen.
2. **Vorschau.** Die App erkennt Spalten (Datum, Betrag, Beschreibung, IBAN) automatisch und lässt das Mapping direkt in der Tabelle korrigieren.
3. **Bereinigung.** Datumsformate werden normalisiert (Schweizer `tt.mm.jjjj` → ISO), Vorzeichen korrigiert, Abkürzungen aufgelöst.
4. **Kontieren.** Die Nutzerin kontiert Zeile für Zeile über eine editierbare Tabelle. Wo aus mandanteneigenen Regeln oder aus früheren Buchungen ein Soll-/Haben-Konto abgeleitet werden kann, wird es vorbelegt und markiert; alles andere wird manuell erfasst oder korrigiert. Der automatische Vorschlags-Layer ist heute begrenzt (§09) und wird schrittweise ausgebaut.
5. **Buchen.** Ein Klick schreibt jede Buchung als *manual entry* via bexio-REST-API.

KERNAUSSAGE

Der Kern von BexFLOW ist der strukturierte Ablauf: Datei einlesen, Spalten sauber mappen, Zeilen bereinigen und in einer editierbaren Tabelle in Sammelbuchungen überführen — mit Idempotenz-Schutz gegen Doppelbuchungen und einer Import-Historie. Vorschläge zur Kontierung (Regeln, Historie, teilweise KI) sind ein optionaler Hilfs-Layer, der aktuell begrenzt ist und schrittweise ausgebaut wird. Die buchhalterische Entscheidung trifft die Nutzerin.

Geschäftsmodell

Abgerechnet wird per **Token**: eine gebuchte Transaktion entspricht einem Token. Neue Nutzer erhalten 50 Test-Token und bis 09.08.2026 zusätzlich 100 Bonus-Credits. Anschliessend werden Starter-, Pro- oder Business-Pläne per Stripe Checkout gebucht (monatlich oder jährlich). Duplikate, die vom Idempotenz-Ledger abgefangen werden, kosten **nie** Token.

Was BexFLOW nicht ist

- Keine vollständige Buchhaltung — bexio bleibt das führende System.
- Keine Belegerkennung oder OCR — Eingabe ist immer ein Bankauszug.
- Kein QR-Rechnungs-Generator und keine Zahlungsauslösung.

Der Ablauf, vom Login bis zur Buchung

Jeder Schritt entspricht einer Seite unter `frontend/app/`:

01 **/login**

E-Mail + Passwort + 6-stelliger E-Mail-Code (OTP). Session-Cookie `bp_auth_sid`.

02 **/settings**

Nutzerin verbindet bexio via OAuth 2.0. Zusätzliches Cookie `bp_bexio_sid`, an den Auth-User gebunden.

03 **/upload**

Datei → POST `/api/imports/analyze-candidates`. Backend parst mit pandas + openpyxl und schlägt Mappings vor.

04 **/preview**

Nutzerin bestätigt das Spalten-Mapping. `/api/ai/import/parse` und `/api/ai/import/assist?stage=preview` laufen.

05 **/cleanup**

Bereinigungslauf schlägt Korrekturen vor; einzeln annehmen oder verwerfen.

06 **/spreadsheet**

Die editierbare Vollansicht. `/api/ai/import/assist?stage=spreadsheet` ruft die komplette 8-Schichten-Pipeline (§09) auf und liefert Soll/Haben-Vorschläge mit Konfidenz und Begründung.

07 **/direct-import**

Buchen in bexio. `/api/bexio/direct-import/post` legt pro Zeile ein `manual_entry` an, im asynchronen Hintergrund-Thread.

08 **/history · /massenmutationen · /bexio-export**

Frühere Importe, Massенbearbeitung/-löschung gebuchter Einträge, Jahres-Journal-Export.

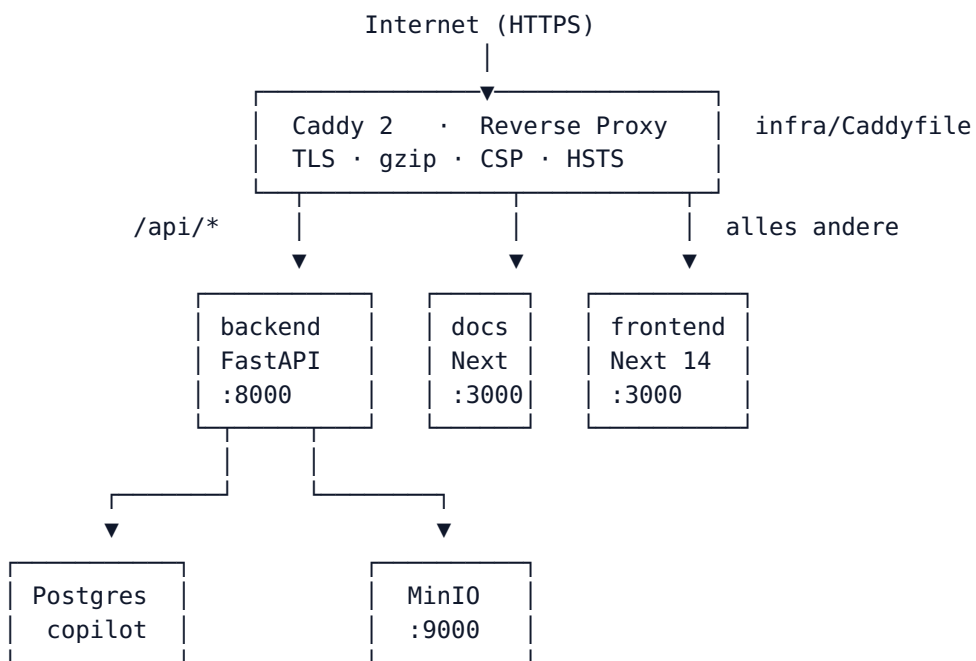
Die kritische Garantie: Idempotenz

Jede Buchung, die bexio erreicht, wird vor dem API-Aufruf mit einem Fingerabdruck versehen:

```
# app/services/posted_entries_store.py
sha256(tenant + date + |amount| + debit + credit + currency + normalized_text)
```

Bei jedem Wiederholungsversuch — Doppelklick, Neustart eines Worker-Threads, Netzwerk-Aussetzer — passt der Fingerabdruck auf eine bestehende Zeile in `posted_entries`. Der zweite Versuch liefert dann Status `DUPLICATE`, statt eine doppelte Buchung im echten Ledger anzulegen. Duplikate werden nicht abgerechnet.

Wer mit wem spricht



extern: bexio API · Infomaniak AI · Stripe · SMTP · BAZG-Devisenkurse

Services und ihre Aufgaben

SERVICE	ROLLE	IMAGE / BUILD
caddy	TLS-Terminierung, gzip, Security-Header, pfadbasiertes Routing (<code>/api/*</code> → Backend, sonst → Frontend). Beherrscht mehrere Hostnamen gleichzeitig.	<code>caddy:2</code>
frontend	Next.js 14 App-Router-UI. Überwiegend clientseitig; React Query spricht <code>/api</code> an.	Build aus <code>frontend/</code>
docs	Separate statische Next.js-Site unter <code>wiki.burkhardt-partners.ch</code> .	Build aus <code>bp-docu/</code>
backend	FastAPI-Monolith. Eine Datei: <code>backend/main.py</code> . Läuft mit uvicorn im Single-Worker-Modus .	Build aus <code>backend/</code>
postgres	Sämtlicher State: Auth, Sessions, verschlüsselte OAuth-Tokens, Import-Historie, Embedding-Cache, Billing, KI-Audit.	<code>postgres:16</code>
minio	S3-kompatibel. Aktuell bereitgestellt, aber nicht genutzt — reserviert für spätere Blob-Ablage.	<code>minio/minio</code>
tasktrack	Interne, nicht zu BexFLOW gehörende Aufgabenverwaltung, die denselben Caddy nutzt. Kann ignoriert werden.	Build aus <code>/home/ubuntu/TaskTrack</code>

HINWEIS

Das Backend ist bewusst ein Ein-Datei-Monolith. Ein Split von `main.py` in APIRouter-Module ist als Folgearbeit erfasst, wurde aber während des Live-Betriebs mit realen Mandanten als zu riskant eingestuft. Kein Bug, sondern eine bewusste Entscheidung.

§04

REPOSITORY

Wo was liegt

```

# /opt/bp-pilot/app/
├─ AGENTS.md          # AI-Agent-Kontext / "Soul File"
├─ backend/
│   └─ Dockerfile
│   └─ requirements.txt # 12 gepinnte Abhängigkeiten, Python 3.11
│   └─ main.py         # 7'988 Zeilen · 65 HTTP-Routen · die App
│   └─ app/
│       └─ db.py        # SQLAlchemy-Engine (Postgres oder SQLite-Fallback)
│       └─ models/canonical.py # kanonische Pydantic-Transaktionstypen
│       └─ services/    # sauber getrennte Businesslogik
│           └─ ai_assist.py # 920 Mapping/Cleanup/Regel-Matcher
│           └─ ai_engine.py # 255 lexikalische Historien-Suche
│           └─ ai_semantic.py # 210 Embedding-basierte Suche
│           └─ ai_booking.py # 284 LLM-Buchungs-Vorschläge
│           └─ ai_parsing.py # 251 LLM-gestütztes Spalten-Parsing
│           └─ ai_provider.py # 462 Infomaniak-HTTP-Client
│           └─ ai_types.py # geteilte Pydantic-Typen
│           └─ ai_audit_store.py # Entscheidungen → ai_audit_events
│           └─ seed_knowledge.py # Kaltstart (Schweizer KMU-Chart)
│           └─ posted_entries_store.py # Idempotenz-Ledger
│           └─ posting_rules_store.py # Mandanten-Regeln Keyword-Konto
│           └─ rule_mining.py # Regel-Kandidaten aus Historie
│           └─ confidence_calibration.py # Beta-Binomial-Kalibrierung
│           └─ history_store.py # import_history-Tabelle
│           └─ embedding_store.py # Vektor-Cache
│           └─ kv_store.py # DB-basiertes Dict für State
│           └─ auth_store.py # Users / Sessions / OTP / Audit
│           └─ billing_store.py # Token-Wallet / Stripe-Dedup
│           └─ crypto.py # Fernet-Verschlüsselung
│           └─ bexio_import.py # CLI-Tool, nicht Hotpath
│           └─ apply_mapping.py # 580 deterministische Transformationen
│           └─ support_store.py # In-App-Support-Tickets
│       └─ cli/
│           └─ ingest_training_data.py # erzeugt seed_knowledge.json
│           └─ evaluate_ai_booking.py # Evaluations-Harness
│           └─ prepare_bexio_import.py # eigenständiges Dry-Run-CLI
│       └─ data/
│           └─ seed_knowledge.json # 93 Keyword-Konto-Zuordnungen, offline extrahiert
│           └─ standard_booking_rules.json
│   └─ tests/          # 27 pytest-Dateien
├─ frontend/
│   └─ Dockerfile
│   └─ package.json    # Next 14 · React 18 · React Query · xlsx
│   └─ middleware.ts    # serverseitige Auth-Sperre privater Routen
│   └─ app/             # Next.js App Router
│       └─ layout.tsx providers.tsx
│       └─ upload/ preview/ cleanup/ spreadsheet/ direct-import/
│       └─ massenmutationen/ bexio-export/ history/ posting-rules/
│       └─ billing/ settings/ support/ landing/

```

```
| | |─ login/ complete/ dashboard/ clients/
| | |─ components/      # shell · ui · stepper · billing-panel · language-switcher
| | |─ lib/              # csrf.ts · i18n.ts
| | └─ tests/           # Playwright-Smoke-Tests
└─ bp-docu/             # separate Marketing-/Hilfe-Site (Next.js Static Export)
└─ infra/
| | └─ docker-compose.prod.yml
| | └─ Caddyfile         # Multi-Host-Proxy + Security-Header
└─ training_data/       # Ausgangsdateien für seed_knowledge.json (offline)
└─ fixtures/bank-statements/ # Testdaten
```

Einstiegspunkt

`backend/main.py` definiert eine einzige `app = FastAPI(lifespan=...)`. Der Lifespan startet `_run_startup_tasks()`: (a) harte Prüfung, dass `AUTH_CODE_PEPPER` ein echtes Produktions-Secret ist; (b) alle Schemata initialisieren; (c) alte E-Mail-Challenges und Audit-Zeilen aufräumen; (d) laut warnen, wenn `APP_ENCRYPTION_KEY` fehlt.

Routen-Übersicht — die 65 Endpunkte, gruppiert

PRÄFIX	ZWECK
<code>/health · /clients</code>	Liveness + Demo-Mandant
<code>/auth/*</code>	Registrierung, Login, OTP, Verifikation, Passwort-Reset, Session, Logout, Konto löschen
<code>/account/*</code>	Profil, Mitglieder, Einladungen
<code>/marketing/email-list</code>	Newsletter-Anmeldung von der Landingpage
<code>/support/tickets</code>	In-App-Support-Tickets (per E-Mail an <code>info@bexflow.ch</code>)
<code>/billing/*</code>	Stripe-Übersicht, Checkout-Session, Customer-Portal
<code>/stripe/webhook</code>	HMAC-verifizierter Webhook, Deduplizierung über <code>billing_store</code>
<code>/bexio/session · /connect · /callback</code>	OAuth-2.0-Handshake
<code>/bexio/accounts · /currencies · /taxes · /context</code>	Read-Through-Cache der bexio-Metadaten
<code>/bexio/manual-entries/search · /bulk-mutate · /bulk-delete</code>	Lesen und Massenbearbeitung gebuchter Einträge
<code>/bexio/direct-import/post · /jobs/:id · /duplicates/clear</code>	Neue Buchungen anlegen + Duplikat-Verwaltung
<code>/bexio/export-package · /jobs/:id · /download</code>	Jahres-Journal-Exports
<code>/imports/analyze-candidates · /history</code>	Upload-Analyse + Import-Historie
<code>/ai/models · /health · /import/parse · /import/assist · /booking/suggest · /calibration</code>	Die gesamte KI-Oberfläche
<code>/posting-rules · /suggestions · /packages/:id</code>	Manuelle und geminte Regeln

Übergreifende Middleware

- **Auth:** `_get_auth_session_or_401()` prüft `bp_auth_sid` gegen `auth_sessions` und aktualisiert `last_seen_at`.
- **CSRF:** `_require_csrf()` vergleicht Cookie `bp_csrf_token` mit Header `X-CSRF-Token` bei jedem State-verändernden Aufruf.
- **Rate-Limits:** DB-basiert (`auth_rate_hits`), mit In-Memory-Fallback. Buckets pro IP oder pro E-Mail je nach Endpunkt.
- **Bexio-Sperre:** `_get_bexio_session_or_401()` auf jeder Route, die bexio anspricht.
- **Billing-Sperre:** `_raise_if_import_tokens_insufficient()` blockiert Buchungen bei leerem Wallet.

Token-Refresh und Verschlüsselung

bexio-Access-Tokens leben 60 Minuten, Refresh-Tokens ~30 Tage.

`_get_valid_access_token()` erneuert transparent kurz vor Ablauf. Beide Tokens liegen in `_bexio_sessions`, einem `KvDict` über `app_kv` in Postgres. Werte werden mit Fernet verschlüsselt, sofern `APP_ENCRYPTION_KEY` gesetzt ist; das `enc:v1:`-Präfix in `crypto.py` erlaubt Rotation und Koexistenz mit älteren Klartext-Zeilen.

Hintergrundjobs

Kein Celery — schlicht `threading.Thread(daemon=True)`. Drei lange laufende Jobtypen:

- **Direct-Import-Posting** — läuft Zeile für Zeile gegen `/manual_entries`. Fortschritt (`processed / ok_count / error_count / dry_run_count`) wird nach jeder Zeile in den Namespace `direct_import_job` von `KvDict` geschrieben. So sieht das Frontend beim Polling einen Live-Fortschrittsbalken und der Zustand überlebt Container-Neustarts.
- **Manual-Entry-Massenmutation** — Spiegelbild-Job für Bearbeiten/Löschen bereits gebuchter Einträge.
- **Export-Pakete** — langlaufende ZIP-Builds von Jahresjournalen in `BEXIO_EXPORT_DIR`.

Idempotenz auf dem Hotpath

```
def _execute_direct_import_post(payload, sid, sess, user_id):
    for row in payload.rows:
        fp = compute_entry_fingerprint(
            tenant_id=tenant, date_iso=row.date, amount=row.amount,
            debit=row.debit, credit=row.credit,
            currency=row.currency, description=row.description,
        )
        if not payload.allow_duplicates:
            if find_posted_entries(engine, tenant_id, [fp]):
                results.append({"status": "DUPLICATE", ...})
                continue # nicht abgerechnet, nicht gesendet
        resp = requests.post(f"{BEXIO_API_V3}/accounting/manual_entries", ...)
        if resp.ok:
            record_posted_entry(engine, tenant_id, fp, bexio_entry_id, ...)
            consume_import_tokens(engine, user_id, 1)
```

Stack

- **Next.js 14** App Router. Kein eigener Server; der Container startet nur `next start -H 0.0.0.0 -p 3000`.
- **React 18** + **@tanstack/react-query 5** für sämtlichen Server-State.
- **Tailwind CSS 3**. Keine Component-Library — die Primitive stehen in `app/components/ui.tsx`.
- **xlsx**-Paket für die client-seitige Vorschau (die verbindliche Parse-Logik läuft weiterhin auf dem Backend).
- **lucide-react** für Icons.

API-Basis-URL

`NEXT_PUBLIC_API_BASE=/api`. Das Frontend spricht nie direkt mit `backend:8000`; jeder Aufruf läuft über Caddys `handle_path /api/*`, das das `/api`-Präfix entfernt und zum Backend proxied.

middleware.ts

Serverseitige Sperre, die vor jeder Anfrage läuft. Nicht angemeldete Nutzer werden von privaten Routen nach `/login` umgeleitet, bereits angemeldete Nutzer von `/login` wieder zurück in die App.

Auth-Modell im Client

Der Session-Zustand ergibt sich aus dem HttpOnly-Cookie `bp_auth_sid`. Da HttpOnly für JS unsichtbar ist, ruft das Frontend einmal `GET /auth/session` auf; die Antwort ist die einzige verlässliche Wahrheit über „bin ich eingeloggt“. Das CSRF-Token liegt im lesbaren Cookie `bp_csrf_token` und wird bei jedem State-verändernden Aufruf als `X-CSRF-Token`-Header mitgesendet — klassisches Double-Submit-Pattern in `frontend/app/lib/csrf.ts`.

Die Arbeitsfläche /spreadsheet

Hier verbringen Nutzerinnen die meiste Zeit. Beim Öffnen läuft:

```
POST /api/ai/import/assist { stage: "spreadsheet", rows: [...] }
```

Die Antwort liefert vier Vorschlagsarrays; die Seite legt sie farbcodiert über die Zeilen (grün akzeptierbar, orange `needs_review`). Nutzerinnen können:

- Einen Vorschlag mit einem Klick annehmen — das POSTet sofort an `/ai/audit/booking-decision`, damit die Kalibrierung lernt.
- Ein anderes Konto tippen → Autocomplete aus dem Kontenplan des Mandanten.
- Aus einem wiederholten Override eine dauerhafte Regel machen: `POST /posting-rules`.

i18n

`frontend/app/lib/i18n.ts` — handgeschriebenes Wörterbuch. Das UI ist Deutsch-first (Schweizer Markt), Englisch als Fallback über `<LanguageSwitcher/>`.

Die Postgres-Tabellen

Sämtlicher State liegt in Postgres. Jedes Service-Modul legt sein eigenes `CREATE TABLE IF NOT EXISTS` in einer `init_*_schema()`-Funktion an, die beim Start läuft — Alembic wird **nicht** eingesetzt.

TABELLE	ZWECK	ZUSTÄNDIG
<code>auth_users</code>	E-Mail, bcrypt-Hash, AGB, verifiziert am	auth_store
<code>auth_email_challenges</code>	OTP-Codes (SHA-256 + Pepper), Versuche, Ablauf	auth_store
<code>auth_sessions</code>	Session-Cookie-ID → User-ID, <code>expires_at</code> , <code>last_seen_at</code>	auth_store
<code>auth_audit_log</code>	Login/Logout/Fehler, IP, User-Agent	auth_store
<code>auth_account_members</code>	Konten mit mehreren Nutzern (Owner + Eingeladene)	auth_store
<code>auth_rate_hits</code>	Rate-Limit-Trefferlog	main.py
<code>account_profiles</code>	Anzeigename, Sprache, Marketing-Opt-in	auth_store
<code>app_kv</code>	Generischer, namensraumbasierter KV-Store, verschlüsselt. Trägt bexio-OAuth-Sessions, Direct-Import-Job-Status, Bulk-Job-Status, Export-Status.	kv_store
<code>import_history</code>	Jeder abgeschlossene Import, Zeilen als JSON	history_store
<code>posting_rules</code>	Mandantenspezifische Keyword → Konto-Regeln (manuell + gemint)	posting_rules_store
<code>posted_entries</code>	Idempotenz-Ledger — Fingerprint + bexio-Entry-ID	posted_entries_store
<code>ai_audit_events</code>	Jede Akzeptanz/Übersteuerung mit Quelle, roher und kalibrierter Konfidenz, Modell, Hashes. Basis für Kalibrierung und Regel-Mining.	ai_audit_store
<code>embedding_cache</code>	<code>text_hash</code> → JSON-Vektor. ~30–50 KB pro Zeile bei 3584 Dimensionen.	

TABELLE	ZWECK	ZUSTÄNDIG
		embedding_store
<code>billing_accounts</code>	Stripe-Customer-ID, Plan, Wallet-Saldo	billing_store
<code>billing_ledger</code>	Jede Token-Gutschrift, Verbrauch, Rückerstattung	billing_store
<code>billing_webhook_events</code>	Bereits verarbeitete Stripe-Event-IDs (Dedup)	billing_store
<code>support_tickets</code>	In-App-Support-Tickets, gespiegelt per E-Mail	support_store

ACHTUNG

Kein Migrations-System vorhanden. Spaltenänderungen erfolgen per ALTER TABLE IF NOT EXISTS ... ADD COLUMN ... innerhalb des jeweiligen `init_*_schema`. Eine NOT-NULL-Spalte auf einer nicht leeren Tabelle braucht einen zweistufigen Deploy: (1) nullable + Backfill, (2) auf NOT NULL verschärfen. Muster im Code: `_ensure_column`.

8.1 bexio OAuth 2.0

Authorization-Code-Flow mit State-Parameter (kein PKCE — bexio verlangt es nicht).

Scopes: `openid profile email offline_access company_profile accounting`.

```
GET /bexio/connect → liefert bexio-auth_url, setzt State-Cookie
Nutzer → bexio.com-Login
GET /bexio/callback → tauscht Code gegen Tokens (POST BEXIO_TOKEN_URL)
                    legt Access + Refresh verschlüsselt in app_kv ab
                    warmt Mandantenkontext (company_profile, Konten)
```

Sämtliche URLs sind per Env überschreibbar (`BEXIO_AUTHORIZE_URL`, `BEXIO_TOKEN_URL`, `BEXIO_API_V2`, `BEXIO_API_V3`) — nützlich für Staging und Mocks.

WELCHE BEXIO-APIS ANGESPROCHEN WERDEN

- **V3** — Buchhaltungs-Entitäten: `/accounts`, `/currencies`, `/taxes`, `/manual_entries` (POST, GET/Suche, PUT/DELETE bulk).
- **V2** — Legacy-Endpunkte für einzelne Report-Exports, die V3 (noch) nicht liefert.

8.2 Infomaniak AI

Infomaniaks OpenAI-kompatibles LLM. Ein einziger Client in `backend/app/services/ai_provider.py`.

INFOMANIAK_AI_API_KEY	Bearer-Token
INFOMANIAK_AI_BASE_URL	z. B. <code>https://api.infomaniak.com/llm/openai/v1</code>
INFOMANIAK_AI_PRODUCT_ID	Infomaniak-spezifische Mandanten-ID in der URL
INFOMANIAK_AI_MODEL	Standard-Chat-Modell
AI_FALLBACK_MODEL	Fallback bei Fehlern des Hauptmodells (Standard <code>swiss-ai/Apertus-70B-Instruct-2509</code>)
AI_EMBEDDING_MODEL	<code>bge_multilingual_gemma2</code> (3584-dim) oder <code>Qwen/Qwen3-Embedding-8B</code> (4096-dim)
INFOMANIAK_AI_RESPONSE_FORMAT	<code>json_object</code> , wenn unterstützt, sonst <code>none</code>

`chat_json()` validiert die Antwort gegen ein übergebenes Pydantic-Schema und wiederholt den Aufruf ohne `response_format=json_object`, falls der Server ihn ablehnt. Jede Request-/Response-Payload wird per SHA-256 gehasht — nur der Hash landet im Audit-Log, nie der Klartext.

8.3 Stripe-Billing

Drei Pläne (Starter / Pro / Business) × zwei Intervalle (Monat / Jahr). Für jede Kombination gibt es eine `STRIPE_*_PRICE_ID`.

```
POST /billing/checkout-session → gehostete Stripe-Checkout-URL
POST /billing/portal-session → gehostete Stripe-Customer-Portal-URL
POST /stripe/webhook → HMAC-verifiziert (STRIPE_WEBHOOK_SECRET),
Toleranz 300 s, Dedup über
billing_webhook_events
```

Erfolgreiches `invoice.paid` schreibt Tokens ins `billing_ledger`; `customer.subscription.*` aktualisiert den Plan. Die Webhook-Dedup ist neben `posted_entries` die einzige Event-ID-basierte Idempotenz in der App.

8.4 SMTP-Mail

Nüchternes `smtplib` gegen `AUTH_SMTP_HOST`. Genutzt für OTP-Codes, Support-Tickets, Marketing-Opt-ins und Einladungslinks für Mehrbenutzerkonten.

8.5 BAZG-Monatsdurchschnittskurse

`BAZG_AVG_MONTH_URL` — der öffentliche XML-Feed der Eidgenössischen Zollverwaltung. Nicht-CHF-Transaktionen werden mit dem BAZG-Monatsdurchschnitt umgerechnet; so entsprechen Buchungen den schweizerischen Steuerregeln. Ein hardcodierter Fallback in `main.py` deckt 01/2024 bis 05/2026 ab, falls der Feed ausfällt.

Vorschlags-Layer (aktuell begrenzt)

ACHTUNG

Aktueller Stand: Die automatische Kontierung ist heute ein optionaler Hilfsbaustein, keine tragende Säule der Anwendung. Nutzerinnen buchen weiterhin manuell in der Spreadsheet-Ansicht; wo aus mandantenspezifischen Regeln oder aus früherer Historie ein Vorschlag ableitbar ist, wird er vorbelegt. LLM-Aufrufe sind konservativ dosiert und werden schrittweise weiterentwickelt.

Der steuernde Endpunkt ist `POST /ai/import/assist` (`backend/main.py:5116`). Er nimmt die bereinigten Zeilen plus den bexio-Kontext des Mandanten (Kontenplan, Steuercodes, letzte Buchungen) entgegen und liefert vier Vorschlagsarrays: *Mapping*, *Cleanup*, *Belegnummer* und **Kontierungsvorschläge**.

Die Kontierungsvorschläge entstehen in bis zu acht kaskadierenden Schichten. Jede füllt nur, was die vorherigen offen gelassen haben: zuerst deterministisch, zuletzt probabilistisch. So bleibt der Layer bei Regeln und Historie sparsam und ruft das LLM nur für wirklich neue Zeilen.

- | | |
|----|---|
| 01 | Deterministische Buchungsregeln
Nutzer-definierte Keyword → Konto. Feste Konfidenz 1.0. |
| 02 | Interner-Übertrag-Detektor
Wenn zwei eigene Bankkonten auf beiden Seiten erscheinen. |
| 03 | Lexikalische Historien-Suche
Token-Überschneidung + SequenceMatcher gegen frühere <code>manual_entries</code> . |
| 04 | Semantische (Embedding-)Suche
Cosine-Ähnlichkeit gegen dasselbe Korpus, nur für offen gebliebene Zeilen. |
| 05 | Text-Match gegen Kontenplan
Direkter Namensvergleich gegen Konten des Mandanten. |
| 06 | LLM-Fallback
Infomaniak-Chat-JSON. Maximal 25 Zeilen pro Aufruf. |
| 07 | Seed-Wissen (Kaltstart)
93 vorbereitete Schweizer-KMU-Zuordnungen, nur wenn alles darüber leer bleibt. |
| 08 | Verrechnungskonto-Fallback
Letztes Auffangnetz, damit keine Zeile unkontiert bleibt. |

9.1 Deterministische Regeln

Nutzerinnen definieren mandantenspezifische Regeln unter `/posting-rules`: „Enthält die Beschreibung *SUVA*, buche gegen Konto 5810.“ Die Regeln liegen in der Tabelle `posting_rules` (`posting_rules_store`). Der Matcher lebt in `ai_assist.posting_rule_suggestions()`. Regeln lassen sich zu Paketen bündeln; ein Standard-KMU-Paket liegt in `backend/app/data/standard_booking_rules.json`.

9.2 Interner Übertrag

`internal_transfer_posting_suggestions()`. Die Gegen-IBAN wird geprüft: gehört sie einem weiteren Bankkonto desselben Mandanten (alle IBANs sind aus dem

bexio-Kontenplan bekannt), wird Konto-gegen-Konto gebucht — mit Konfidenz 0.98. Das eliminiert die klassische Doppelbuchung firmeninterner Überträge.

9.3 Lexikalische Historien-Suche

`ai_engine.historical_retrieval_suggestions()`. Die 100 jüngsten Importe des Mandanten plus bis zu 1000 `manual_entries` aus den letzten 365 Tagen werden geladen und als `HistoricalBookingExample` übergeben. Die Bewertung kombiniert vier Signale:

SIGNAL	GEWICHT	BEDEUTUNG
Exakter Textmatch	+0.62	<code>norm(desc) == norm(prev_desc)</code>
Treffer Referenznummer	+0.22	Referenz erscheint in einer früheren Buchung
Token-Überschneidung	$\leq +0.24$	Jaccard-artige Abdeckung
Fuzzy-Verhältnis	Tie-Break	<code>difflib.SequenceMatcher</code> auf ersten 600 Zeichen

Der beste Wert gewinnt, sofern er die Schwelle übersteigt. Das Gegenkonto — die Seite, die *nicht* das Bankkonto ist — wird vorgeschlagen. Begründungs-Strings werden mitgeliefert, damit das UI das *Warum* anzeigen kann.

9.4 Semantische Suche

`ai_semantic.py`. Läuft nur, wenn `AI_EMBEDDING_MODEL` gesetzt und `AI_EMBEDDING_RETRIEVAL_ENABLED` aktiv ist.

1. Jede offen gebliebene Zeile wird zu einem Query-String verdichtet (Beschreibung + Referenz + Auftraggeber + Begünstigter + IBAN).
2. Jede historische Beispielbuchung wird analog verdichtet.
3. Beides wird via `provider.embed()` in Vektoren übersetzt — aktuell Infomaniaks `bge_multilingual_gemma2` (3584 Dimensionen). Die Vektoren werden in `embedding_cache` nach SHA-256 des Textes gespeichert; erneute Anfragen sind kostenlos.
4. Der beste Treffer über der Ähnlichkeits-Schwelle wird vorgeschlagen; Konfidenz skaliert mit dem Cosinus.

Diese Schicht ist der grösste Genauigkeitsgewinn gegenüber reiner Lexikalsuche. Sie findet Umbenennungen wie „MIGROS GENOSSENSCHAFT" ↔ „M-

Budget Migros", die Schicht 3 verpasst. Fail-safe: Bei jedem Provider-Fehler bleiben die Ergebnisse aus Schicht 3 unverändert bestehen.

9.5 Text-Match gegen Kontenplan

`chart_account_text_suggestions()`. Substring-Vergleich der Beschreibungs-Tokens gegen die **Namen** im Kontenplan. Trifft Fälle wie „Zinsertrag" → 6800 „Zinsertrag" ohne vorherige Buchung. Niedrige Konfidenz.

9.6 LLM-Fallback

Nur die nach den Schichten 1–5 noch offenen Zeilen werden ans LLM geschickt — nie mehr als `AI_BOOKING_LLM_MAX_ROWS` (Standard 25) pro Aufruf. Der Aufruf erfolgt via `ai_booking.request_booking_suggestions()` und `chat_json()` gegen ein striktes `BookingSuggestion`-Pydantic-Schema. Prompt-Inhalt:

- Vollständiger Kontenplan des Mandanten (JSON: Name + Nummer + MwSt-Typ).
- Bis zu 20 ähnlichste historische Beispiele als Few-Shot.
- Gültige Steuercodes.
- Die offenen Zeilen (id · Datum · Beschreibung · Betrag · Währung · Richtung).

Jeder Vorschlag unterhalb `AI_BOOKING_CONFIDENCE_THRESHOLD` (Standard 0.80) wird als `needs_review` markiert. Kosten, Latenz und SHA-256-Hashes von Request/Response landen in `ai_audit_events` — niemals der Klartext, so bleiben PII draussen.

9.7 Seed-Wissen (Kaltstart)

`seed_knowledge.py`. Aufgebaut aus 16'253 realen Journalzeilen von fünf anonymisierten Mandanten (`training_data/`) und offline zu 93 verallgemeinerbaren Schweizer-KMU-Keywrd-zu-Konto-Zuordnungen verdichtet, via CLI `backend/app/cli/ingest_training_data.py`.

Feuert nur, wenn nichts darüber die Zeile gelöst hat. Vorschläge werden auf Konten be-

grenzt, die im Kontenplan des Mandanten tatsächlich existieren, und tragen immer

`needs_review=true`, weil generisch. Aktivierung über

`AI_SEED_KNOWLEDGE_ENABLED`.

9.8 Verrechnungskonto-Fallback

`fallback_counter_account_suggestions()`. Hat eine Zeile immer noch kein Gegenkonto, schlägt die App das Klärungs-/Verrechnungskonto des Mandanten vor (typischerweise 1099 oder konfigurierbar). So bleibt keine Zeile unbuchbar.

9.9 Konfidenz-Kalibrierung

`confidence_calibration.py`. Nach Zusammenstellung der Vorschläge wird jede handgesetzte Konfidenz **zur empirisch beobachteten Akzeptanzquote pro Quelle** aus `ai_audit_events` hin geschrumpft. Die Formel ist ein Beta-Binomial-Posterior:

```
calibrated = (accepted + α) / (total + α + β)  # α, β wirken als Prior
```

Wurde eine Quelle (z. B. `historical_retrieval`) 80 von 100 Malen akzeptiert, zieht die nächste Konfidenz gegen 0.80 — unabhängig von der hardcodierten Konstanten. Fällt die kalibrierte Konfidenz unter die Review-Schwelle, wird `needs_review` aktiviert. Pro Mandant 300 s gecached. Rohzahlen liefert der Endpunkt `/ai/calibration`.

Woher der Audit-Trail kommt

Jede Akzeptanz und jeder Override wird via `POST` `/ai/audit/booking-decision` nach `ai_audit_events` geschrieben. Diese Daten speisen zwei Dinge: (a) die oben genannte Kalibrierung und (b) den Regel-Mining-Endpunkt `GET` `/posting-rules/suggestions`, der aus „Nutzerin hat für Beschreibungen mit STROMKONTO 12-mal von 13 das Konto 6410 akzeptiert“ einen Vorschlag für eine dauerhafte Regel macht, die per Klick übernommen werden kann.

Grenzen des Vorschlags-Layers heute

- Regeln, Historie und Vorschläge greifen zuverlässig erst mit ausreichenden Vorlaufdaten pro Mandant.
- Die Konfidenzwerte der Schichten 1–5 sind noch handgesetzt; die Kalibrierung überlagert sie, ersetzt sie aber nicht.
- Der LLM-Layer läuft konservativ: Zeilenlimit pro Aufruf, fest verdrahteter Fallback auf ein Klärungskonto.
- Der Lernkreislauf ist einseitig — geminte Regeln werden vorgeschlagen, aber nicht automatisch übernommen.

- Kontrolle und Verantwortung liegen bei der Nutzerin; die Vorschläge sind Hilfestellung, keine Freigabe.

TRANSPORT

- Caddy stellt Let's-Encrypt-Zertifikate automatisch aus. HSTS
`max-age=31536000; includeSubDomains; preload`.
- CSP `frame-ancestors 'none'; base-uri 'self'; object-src 'none'`.
- X-Frame-Options DENY · X-Content-Type-Options nosniff · strikte Referrer-Policy.

AUTHENTIFIZIERUNG

- Passwörter: `bcrypt`, Standard-Workfaktor.
- Sessions: 12 Stunden TTL (`AUTH_SESSION_TTL_HOURS`), HttpOnly, über HTTPS Secure, SameSite=Lax.
- Login rate-limited pro IP und pro E-Mail. Fehlversuche verraten nicht, ob die E-Mail existiert (Antworten sind zeitkonstant).
- OTP: 6-stelliger Code, SHA-256 + `AUTH_CODE_PEPPER`, 10 Minuten TTL, max. 6 Versuche, 45 s Resend-Cooldown.
- Der Startup **bricht hart ab**, wenn `AUTH_CODE_PEPPER` fehlt, „change_me“ enthält oder kürzer als 32 Zeichen ist.

CSRF

Double-Submit: lesbares Cookie `bp_csrf_token` gegen Header `X-CSRF-Token`, geprüft von `_require_csrf()` auf jeder State-verändernden Route. Bei Abweichung 403.

BEXIO-SESSION-BINDUNG

`_bexio_session_matches_auth()` — eine bexio-Session ist an die Auth-`user_id` gebunden, die sie angelegt hat. Wechselt das Auth-Cookie den Nutzer, wird die bexio-Session beim nächsten Request verworfen. So werden Cross-User-IDOR-Angriffe auf bexio-Kontexte verhindert. Test: `tests/test_security_bexio_session_binding.py`.

VERSCHLÜSSELUNG IM RUHEZUSTAND

bexio-Access- und Refresh-Tokens, OAuth-State-Records und alle verschlüsselten KV-Werte werden mit Fernet und `APP_ENCRYPTION_KEY` verschlüsselt. Mehrere Schlüssel für Rotation sind unterstützt (mit Komma getrennt → `MultiFernet`). Fehlender Key → die App läuft, warnt aber laut beim Start und speichert Klartext.

UPLOAD-SICHERHEIT

- Harte Obergrenze `MAX_UPLOAD_BYTES` (Standard 10 MiB).
- `openpyxl` wird mit Defaults aufgerufen; die Upload-Grenze begrenzt Zip-Bomb-Risiko.
- Dateityp-Validierung via `chardet` und Magic Bytes.

WEBHOOKS

Der Stripe-Webhook verifiziert die HMAC-Signatur mit 300 s Toleranz und dedupliert über die Event-ID. Es ist der einzige eingehende Webhook.

Voraussetzungen

- Docker + Docker-Compose-Plugin (Compose v2).
- Python 3.11 (nur, wenn das Backend ausserhalb Docker läuft).
- Node 18+ (nur, wenn das Frontend ausserhalb Docker läuft).
- Eine **bexio-Developer-App** mit OAuth-Client-ID/Secret. Registrierung unter `developer.bexio.com`. Als erlaubte Redirect-URI: `http://localhost/api/bexio/callback`.
- Ein **Infomaniak-AI-Produkt** mit API-Key.
- Optional: Stripe-Testkonto für die Billing-Endpunkte.

Minimale .env

Datei `/opt/bp-pilot/app/.env` erstellen (aus `infra/docker-compose.prod.yml` als `../.env` referenziert). Vollständige Liste in §13. Minimaler Startsatz:

```
# --- Basis ---
APP_ENV=dev
DOMAIN=localhost
CORS_ORIGINS=http://localhost,https://localhost
DATABASE_URL=postgresql://copilot:devpass@postgres:5432/copilot
POSTGRES_PASSWORD=devpass

# --- Verschlüsselung + Auth ---
APP_ENCRYPTION_KEY=<python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key())">
AUTH_CODE_PEPPER=<python -c "import secrets; print(secrets.token_urlsafe(48))">
AUTH_APP_BASE_URL=http://localhost

# --- Mail (dev: Mailhog / Mailtrap) ---
MAIL_HOST=mailhog
MAIL_PORT=1025
MAIL_ENCRYPTION=none
MAIL_FROM_ADDRESS=no-reply@localhost

# --- bexio OAuth ---
BEXIO_CLIENT_ID=...deine Dev-Client-ID...
BEXIO_CLIENT_SECRET=...
BEXIO_REDIRECT_URI=http://localhost/api/bexio/callback

# --- Infomaniak AI ---
INFOMANIAK_AI_API_KEY=...
INFOMANIAK_AI_BASE_URL=https://api.infomaniak.com/llm/openai/v1
INFOMANIAK_AI_PRODUCT_ID=...
INFOMANIAK_AI_MODEL=mistral-nemo-instruct-2407
AI_EMBEDDING_MODEL=bge_multilingual_gemma2
AI_EMBEDDING_RETRIEVAL_ENABLED=1
AI_SEED_KNOWLEDGE_ENABLED=1
AI_BOOKING_CONFIDENCE_THRESHOLD=0.80

# --- Stripe (Testmodus) ---
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_PUBLISHABLE_KEY=pk_test_...

# --- MinIO (Dev-Defaults reichen) ---
MINIO_ROOT_USER=minio
MINIO_ROOT_PASSWORD=minio12345
```

Stack hochfahren

```
cd /opt/bp-pilot/app/infra
docker compose -f docker-compose.prod.yml up -d --build
open http://localhost
```

Nur Backend, mit Hot-Reload

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export $(grep -v '^#' ../.env | xargs)
uvicorn main:app --reload --port 8000
```

Nur Frontend

```
cd frontend
npm install
NEXT_PUBLIC_API_BASE=http://localhost:8000 npm run dev
```

Erster Smoke-Test

1. `curl http://localhost/api/health` → `{"status":"ok"}`
2. Auf `/login` registrieren; die OTP landet in Mailhog.
3. Auf `/settings` bexio verbinden.
4. `fixtures/bank-statements/*.csv` auf `/upload` hochladen.
5. Auf `/spreadsheet` prüfen: es sollten Vorschläge mit gemischten Quellen sichtbar sein (`historical_retrieval`, `rule`, `chart_name`).
6. Auf `/direct-import` Dry-Run mit `dry_run=true`.

Deployment auf den Live-Server

Wo Prod läuft

- **Server:** Infomaniak Ubuntu-VM (`bp-pilot-vm`).
- **Pfad:** `/opt/bp-pilot/app`.
- **Deploy-Branch:** `main`.

Standard-Deploy

```
ssh ubuntu@app.bexflow.ch
cd /opt/bp-pilot/app
git fetch origin
git checkout main && git pull --ff-only
cd infra
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
curl -fsS https://app.bexflow.ch/api/health
```

ACHTUNG

Der Build-Kontext ist ein Verzeichnis, nicht ein Git-Commit. Alles, was in `/opt/bp-pilot/app` uncommitted liegt, wird mitdeployt. Vor jedem Deploy auf dem Server `git status` ausführen und offene Änderungen committen oder stash en. Bekanntes Rollback-Risiko.

Rollback

```
git log --oneline -20
git checkout <letzter_guter_sha>
docker compose -f infra/docker-compose.prod.yml up -d --build
```

Zero-Downtime-Hinweise

- `uvicorn` läuft absichtlich **single-worker**: eine echte Cross-Worker-Kohärenz bei bexio-Token-Refreshes bräuchte Redis, das nicht im Stack ist. Ein Rolling Restart schneidet hart, meist unter 3 s.
- Langlaufende Direct-Import- und Bulk-Mutation-Jobs sterben beim Restart. Ihr Zustand ist in `app_kv` gesichert, Status-Endpunkte liefern also den letzten Snapshot — die Jobs starten aber nicht von selbst neu. Restarts ausserhalb der Geschäftszeiten planen.

Datenbank-Backups

Verantwortung der Betreiber. Das Volume `pg_data` lässt sich mit folgendem Befehl sichern:

```
docker exec bp-pilot-postgres-1 pg_dump -U copilot copilot \  
| gzip > backup-$(date +%F).sql.gz
```

Basis / Infrastruktur

APP_ENV	dev / staging / prod. Informativ.
DOMAIN	Primäre Domain für Caddy. Standard <code>app.bexflow.ch</code> .
CORS_ORIGINS	Komma-getrennte Origins für FastAPI-CORS.
DATABASE_URL	SQLAlchemy-URL. Fallback: <code>sqlite:///imports.db</code> .
POSTGRES_PASSWORD	Wird beim ersten Start ins Postgres-Image geschrieben.
MAX_UPLOAD_BYTES	Harte Obergrenze pro Upload. Standard 10 MiB.

Verschlüsselung / Auth

APP_ENCRYPTION_KEY	Komma-getrennte Fernet-Keys. Der erste verschlüsselt; alle werden zum Entschlüsseln probiert. Ohne Key wandern Tokens im Klartext in die DB.
AUTH_CODE_PEPPER	≥ 32 Zeichen, kein „change_me“. Beim Start verpflichtend geprüft.
AUTH_SESSION_TTL_HOURS	Session-Lebensdauer. Standard 12.
AUTH_OTP_TTL_MINUTES · MAX_ATTEMPTS · RESEND_COOLDOWN_SECONDS	OTP-Parameter (10 / 6 / 45).
AUTH_APP_BASE_URL	Für ausgehende Links in E-Mails.
AUTH_EMAIL_FROM · _NAME	Absender der Auth-Mails.
AUTH_SMTP_HOST · PORT · USER · PASS · TLS	SMTP für Auth-Mails. Fällt auf <code>MAIL_*</code> zurück.
MAIL_HOST · PORT · USERNAME · PASSWORD · ENCRYPTION · FROM_ADDRESS · FROM_NAME	App-weite SMTP-Defaults.

bexio

BEXIO_CLIENT_ID · BEXIO_CLIENT_SECRET	OAuth-App-Credentials.
BEXIO_REDIRECT_URI	Callback-URL. Standard <code>{base}/api/bexio/callback</code> .
BEXIO_SCOPES	OAuth-Scopes. Deckt OpenID + Accounting ab.
BEXIO_AUTHORIZE_URL · TOKEN_URL	Für Staging überschreibbar.
BEXIO_API_V2 · V3	Basis-URLs. Standard bexio.com.
BEXIO_OAUTH_STATE_TTL_SECONDS	TTL des State-Records. Standard 900.
BEXIO_EXPORT_DIR	Zielordner für Jahres-ZIPs. Standard <code>/tmp/bexio_exports</code> .
BEXIO_EXPORT_MAX_FILES · CONSERVATIVE_DELAY_SECONDS	Rate-/Volumen-Regler für den Export.

KI

INFOMANIAK_AI_API_KEY	Bearer-Token.
INFOMANIAK_AI_BASE_URL	OpenAI-kompatible Basis-URL.
INFOMANIAK_AI_PRODUCT_ID	Infomaniak-Mandanten-ID.
INFOMANIAK_AI_MODEL	Standard-Chat-Modell.
INFOMANIAK_AI_RESPONSE_FORMAT	<code>json_object</code> oder <code>none</code> .
AI_FALLBACK_MODEL	Fallback bei Fehlern des Hauptmodells.
AI_EMBEDDING_MODEL	Aktiviert die semantische Schicht. Aktuell <code>bge_multilingual_gemma2</code> .
AI_EMBEDDING_RETRIEVAL_ENABLED	Kill-Switch für die semantische Suche.
AI_SEED_KNOWLEDGE_ENABLED	Kill-Switch für den Kaltstart-Layer.
AI_BOOKING_CONFIDENCE_THRESHOLD	Standard 0.80 — darunter → <code>needs_review</code> .
AI_BOOKING_LLM_MAX_ROWS	Zeilenlimit pro LLM-Aufruf. Standard 25.
AI_BOOKING_MAX_BATCH_SIZE	LLM-Batch-Grösse (auf [1,100] beschränkt).
AI_CALIBRATION_TTL_SECONDS	Cache-Lebensdauer der Kalibrierung. Standard 300.
AI_BOOKING_SUGGESTIONS_ENABLED	Master-Schalter für die LLM-Schicht.

Stripe

STRIPE_SECRET_KEY · PUBLISHABLE_KEY · WEBHOOK_SECRET	Standard-Stripe-API-Keys.
STRIPE_WEBHOOK_TOLERANCE_SECONDS	Toleranz für den Signatur-Zeitstempel. Standard 300.
STRIPE_STARTER · PRO · BUSINESS_PRODUCT_ID	Stripe-Product-IDs.
STRIPE_{PLAN}_{MONTH, YEAR}_PRICE_ID	Sechs Price-IDs: Plan × Intervall.
BILLING_TRIAL_TOKENS	Gratis-Tokens bei Registrierung. Standard 50.
SIGNUP_BONUS_CREDITS · CUTOFF_DATE · TIMEZONE	Zeitlich befristeter Launch-Bonus.

Betrieb

SUPPORT_TICKET_EMAIL · FORWARD_EMAIL	Support-Postfach und Weiterleitung.
SUPPORT_TICKET_MAX_ATTACHMENT_BYTES · ATTACHMENTS	Anhangs-Limits. Standard 8 MiB × 5.
MARKETING_EMAIL_LIST_EMAIL · CC_EMAIL	Empfänger des Landing-Opt-ins.
BAZG_AVG_MONTH_URL	FX-Quelle. Fallback-Dict deckt 01/2024...05/2026 ab.

Backend

```
cd backend
pytest -q
```

27 pytest-Dateien decken ab: Auth & User-Enumeration, CSRF, Rate-Limits, Stripe-Webhook, sicherheitsorientierte Upload-Prüfungen, bexio-Session-Bindung, IDOR beim Account-Löschen, Crypto, KV-Verschlüsselung, KI-Parsing/-Booking/-Semantik, Duplikat-Ledger, Konfidenz-Kalibrierung, Regel-Store, Regel-Mining, Seed-Wissen, Marketing-Mailliste. Tests laufen mit SQLite (In-Memory oder Tempfile) — keine echte DB nötig.

Frontend

```
cd frontend
npm run test:import-engine      # Node-Harness – Golden-CSV → erwartete Zeilen
npm run smoke:live              # Playwright-Smoke gegen eine Live-URL
```

End-to-End-Fixtures

`fixtures/bank-statements/` enthält bereinigte CSV-/XLSX-Dateien von UBS, PostFinance, Raiffeisen und Revolut zur Reproduktion von Parse-Bugs.

Bekannte Themen und Follow-Ups

Verschobene technische Arbeit

- `backend/main.py` aufteilen in `APIRouter`-Dateien (Auth / bexio / KI / Billing / Manual-Entries / Imports). ~8 kLoC in einer Datei sind handhabbar, aber nicht dauerhaft ideal.
- **Redis für Cross-Worker-Token-Refresh-Kohärenz.** Bis dahin bleibt uvicorn in Prod single-worker.
- **Alembic-Migrationen.** Jede Schemaänderung heute per `CREATE TABLE IF NOT EXISTS` oder `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`. Für Additiv-Änderungen okay, für alles andere unangenehm.
- **Deploys an einen Git-SHA binden.** Aktuell ist der Build-Kontext das Arbeitsverzeichnis — uncommitted Code wandert mit in Prod. Abhilfe: CI-gebaute Images oder `git archive` im Build.
- **`embedding_cache`-Wachstum.** ~30–50 KB pro Zeile bei 3584 Dimensionen als JSON. Ab ~100 k Transaktionen pro Mandant sollte eine grössenbasierte Verdrängung her.

Produkt-Follow-Ups

- **Lernkreislauf.** Jede Nutzer-Überschreibung wird auditiert, geminte Regeln (`rule_mining.py`) werden aber nur vorgeschlagen, nicht automatisch übernommen. Auto-Adoption ab N Akzeptanzen steht UX-seitig noch aus.
- **Konfidenz-Konstanten** in den Schichten 1–5 sind noch handgesetzt; die Kalibrierung überlagert sie, ersetzt sie aber nicht.

Operative Besonderheiten

- Der Service `tasktrack` in `docker-compose.prod.yml` ist ein anderes internes Tool. Wer nur BexFLOW verantwortet, kann ihn aus dem Compose-Aufruf entfernen.
- `bp-docu` serviert die Marketing-Site unter `wiki.burkhart-partners.ch`. Static Export; Inhaltsänderungen erfordern einen Rebuild.
- Support-Tickets werden nach `sick@burkhart-partners.ch` gespiegelt — die Inbox des Gründers. Bei Handover `SUPPORT_TICKET_FORWARD_EMAIL` anpassen.

„Die KI schlägt nichts vor“

1. `curl https://app.bexflow.ch/api/ai/health` — sollte die konfigurierten Modelle zurückgeben.
2. Prüfen, ob `INFOMANIAK_AI_API_KEY` gesetzt und gültig ist.
3. Nur die semantische Schicht tot?
`AI_EMBEDDING_MODEL=bge_multilingual_gemma2` exakt so schreiben — jede andere Schreibweise / jeder Anzeigename führt bei Infomaniaks `/embeddings` zu 404.

„Ein Nutzer bekommt bei jeder Zeile DUPLICATE“

Das Dedup-Ledger arbeitet korrekt — die Fingerabdrücke passen auf frühere Buchungen. Entweder lädt die Nutzerin denselben Auszug erneut (erwartet), oder der Fingerabdruck ist zu breit. Notausgang: beim Direct-Import-POST

`allow_duplicates=true` setzen (im UI als „Duplikate zulassen“ sichtbar) oder gezielt Fingerabdrücke via `POST /bexio/direct-import/duplicates/clear` löschen.

„Ein Nutzer ist aus bexio ausgeloggt“

Erwartet nach ~30 Tagen (Refresh-Token-Lebensdauer). Reconnect über `/bexio/connect?reconnect=true`.

„Stripe-Webhook-Events werden nicht verarbeitet“

1. Prüfen, ob `STRIPE_WEBHOOK_SECRET` mit dem Endpoint-Secret im Stripe-Dashboard übereinstimmt.
2. In `billing_webhook_events` nach der Event-ID suchen — steht sie drin, ist der Dedup ordnungsgemäss aktiv.
3. Zeitverschiebung: Toleranz ist 300 s; `ntpdate`, falls die Serveruhr driftet.

„Container startet nicht: AUTH_CODE_PEPPER“

Der Start bricht absichtlich hart ab, wenn der Pepper fehlt, zu kurz ist oder „change_me“ enthält. Env korrigieren und neu starten.

Notfall: letzten Deploy zurückrollen

```
cd /opt/bp-pilot/app
git log --oneline -5
git checkout <letzter_sha>
docker compose -f infra/docker-compose.prod.yml up -d --build backend
curl -fsS https://app.bexflow.ch/api/health
```

Wo zuerst suchen, wenn etwas komisch ist

SYMPTOM	BLICK ZUERST HIERHER
Buchung falsch kontiert	<code>ai_audit_events</code> für die Zeile → welche Schicht / welches Modell hat sie erzeugt
Buchung bei bexio fehlgeschlagen	Direct-Import-Job-Status: <code>GET /bexio/direct-import/jobs/:id</code>
Nutzer kann sich nicht einloggen	<code>auth_audit_log</code> nach E-Mail filtern
Zahlung nicht gutgeschrieben	<code>billing_ledger</code> nach <code>billing_account_id</code>
Import unleserlich	<code>import_history.payload_json</code> — die geparsten Rohzeilen liegen archiviert vor

KERNAUSSAGE

Dieses Dokument deckt lokalen Betrieb, Deployment, die wichtigsten Module und das Runbook für den Live-Betrieb ab. Für Details, die hier nicht ausgeführt sind, ist der primäre Anlaufpunkt `backend/main.py`, gefolgt von den Modulen unter `backend/app/services/`.

